Multi-Period Optimization with Optuna: `Python + Optuna + PnL + Volume Constraints`

June 3, 2025

Overview

This document describes a Python implementation for Multi-Period Optimization (MPO) using Optuna. The goal is to optimize two objectives:

- Maximize Profit and Loss (PnL)
- Maximize Executed Volume

Additional features include:

- Early pruning of bad trials (low volume or negative PnL)
- Intermediate progress reporting
- Saving results and Optuna study for reuse

Installation Requirements

```
pip install optuna pandas
```

Python Code

File: `mpo_optuna_optimizer.py`

```python
import optuna
import random
from typing import Tuple, Optional, Callable
import pandas as pd

class BaseConstraintPruner:
    def __init__(self, min_volume: float, min_pnl: float):
        self.min_volume = min_volume
        self.min_pnl = min_pnl

    def check(self, trial: optuna.Trial, pnl: float, volume: float):
        if pnl < self.min_pnl:
            trial.set_user_attr("prune_reason", f"PnL {pnl:.2f} < {self.min_pnl}")
            raise optuna.TrialPruned()
        if volume < self.min_volume:
            trial.set_user_attr("prune_reason", f"Volume {volume:.1f} < {self.min_volume}")
            raise optuna.TrialPruned()

class MPOOptimizer:
    def __init__(self, min_volume: float = 1000, target_volume: float = 10000):
```

```
        self.min_volume = min_volume
        self.target_volume = target_volume

    def run_simulation(self, aggression: float, spread: float, trial: optuna.Trial, steps:
    int = 100) -> Tuple[float, float]:
        pnl = 0.0
        executed = 0.0

        for step in range(steps):
            available = max(100, 1000 * (1 - aggression))
            step_volume = min(self.target_volume - executed, available * aggression)
            executed += step_volume

            price_move = random.gauss(0, 0.1)
            pnl += step_volume * (price_move - (0.05 * spread))

            trial.report(executed, step)
            if trial.should_prune():
                trial.set_user_attr("prune_reason", f"Pruned at step {step} (volume={
    executed:.1f})")
                raise optuna.TrialPruned()

            if executed >= self.target_volume:
                break

        final_pnl = pnl - (0.1 * aggression * executed)
        return final_pnl, executed

    def optimize(self, n_trials: int = 100, study_name: str = "mpo_study"):
        study = optuna.create_study(
            directions=["maximize", "maximize"],
            study_name=study_name,
            storage=f"sqlite:///{study_name}.db",
            load_if_exists=True,
            sampler=optuna.samplers.TPESampler()
        )

        constraint_checker = BaseConstraintPruner(min_volume=self.min_volume, min_pnl=0.0)

        def objective(trial):
            aggression = trial.suggest_float("aggression", 0.1, 1.0)
            spread = trial.suggest_float("spread", 0.5, 2.0)

            pnl, volume = self.run_simulation(aggression, spread, trial)
            constraint_checker.check(trial, pnl, volume)

            return pnl, volume

        study.optimize(objective, n_trials=n_trials)

        df = study.trials_dataframe()
        df.to_csv(f"{study_name}_results.csv", index=False)
        return study

if __name__ == "__main__":
    optimizer = MPOOptimizer(min_volume=2000, target_volume=10000)
    study = optimizer.optimize(n_trials=50)
    best = study.best_trials[0]
    print(f"Best PnL: {best.values[0]:.2f}, Volume: {best.values[1]:.0f}")
    print(f"Parameters: {best.params}")
```

Output Files

- `mpo_study.db` - Optuna study database (SQLite)
- `mpo_study_results.csv` - CSV file of all trials

Optional Visualizations in Jupyter Notebook

```
import optuna.visualization as vis
vis.plot_pareto_front(study).show()
vis.plot_param_importances(study).show()
```

Customization Suggestions

- Add slippage models or market impact in ‘run$_s imulation$‘$Addmorecomplexobjectiveslike Sharperatioordrawdown$
- Replace random simulation with a CVXPY optimizer for real MPO logic
- Create additional pruners by subclassing ‘BaseConstraintPruner‘ for customized behavior

article listings color hyperref amsmath graphicx caption geometry margin=1in
Multi-Period Optimization with Optuna: `Python + Optuna + PnL + Volume Constraints` June 3, 2025

Overview

This document describes a Python implementation for Multi-Period Optimization (MPO) using Optuna. The goal is to optimize two objectives:

- Maximize Profit and Loss (PnL)
- Maximize Executed Volume

Additional features include:

- Early pruning of bad trials (low volume or negative PnL)
- Intermediate progress reporting
- Saving results and Optuna study for reuse

Installation Requirements

```
pip install optuna pandas
```

Python Code

File: `mpo_optuna_optimizer.py`

```
import optuna
import random
from typing import Tuple, Optional, Callable
import pandas as pd

class BaseConstraintPruner:
    def __init__(self, min_volume: float, min_pnl: float):
        self.min_volume = min_volume
        self.min_pnl = min_pnl

    def check(self, trial: optuna.Trial, pnl: float, volume: float):
        if pnl < self.min_pnl:
            trial.set_user_attr("prune_reason", f"PnL {pnl:.2f} < {self.min_pnl}")
```

```
            raise optuna.TrialPruned()
        if volume < self.min_volume:
            trial.set_user_attr("prune_reason", f"Volume {volume:.1f} < {self.min_volume}")
            raise optuna.TrialPruned()

class MPOOptimizer:
    def __init__(self, min_volume: float = 1000, target_volume: float = 10000):
        self.min_volume = min_volume
        self.target_volume = target_volume

    def run_simulation(self, aggression: float, spread: float, trial: optuna.Trial, steps:
    int = 100) -> Tuple[float, float]:
        pnl = 0.0
        executed = 0.0

        for step in range(steps):
            available = max(100, 1000 * (1 - aggression))
            step_volume = min(self.target_volume - executed, available * aggression)
            executed += step_volume

            price_move = random.gauss(0, 0.1)
            pnl += step_volume * (price_move - (0.05 * spread))

            trial.report(executed, step)
            if trial.should_prune():
                trial.set_user_attr("prune_reason", f"Pruned at step {step} (volume={
    executed:.1f})")
                raise optuna.TrialPruned()

            if executed >= self.target_volume:
                break

        final_pnl = pnl - (0.1 * aggression * executed)
        return final_pnl, executed

    def optimize(self, n_trials: int = 100, study_name: str = "mpo_study"):
        study = optuna.create_study(
            directions=["maximize", "maximize"],
            study_name=study_name,
            storage=f"sqlite:///{study_name}.db",
            load_if_exists=True,
            sampler=optuna.samplers.TPESampler()
        )

        constraint_checker = BaseConstraintPruner(min_volume=self.min_volume, min_pnl=0.0)

        def objective(trial):
            aggression = trial.suggest_float("aggression", 0.1, 1.0)
            spread = trial.suggest_float("spread", 0.5, 2.0)

            pnl, volume = self.run_simulation(aggression, spread, trial)
            constraint_checker.check(trial, pnl, volume)

            return pnl, volume

        study.optimize(objective, n_trials=n_trials)

        df = study.trials_dataframe()
        df.to_csv(f"{study_name}_results.csv", index=False)
        return study

if __name__ == "__main__":
    optimizer = MPOOptimizer(min_volume=2000, target_volume=10000)
    study = optimizer.optimize(n_trials=50)
    best = study.best_trials[0]
    print(f"Best PnL: {best.values[0]:.2f}, Volume: {best.values[1]:.0f}")
    print(f"Parameters: {best.params}")
```

Output Files

- `mpo_study.db` - Optuna study database (SQLite)
- `mpo_study_results.csv` - CSV file of all trials

Optional Visualizations in Jupyter Notebook

```
import optuna.visualization as vis
vis.plot_pareto_front(study).show()
vis.plot_param_importances(study).show()
```

Customization Suggestions

- Add slippage models or market impact in ‘run$_s$*imulation*‘*Addmorecomplexobjectiveslike*$Sharperatioordrawdown$
- Replace random simulation with a CVXPY optimizer for real MPO logic
- Create additional pruners by subclassing ‘BaseConstraintPruner‘ for customized behavior